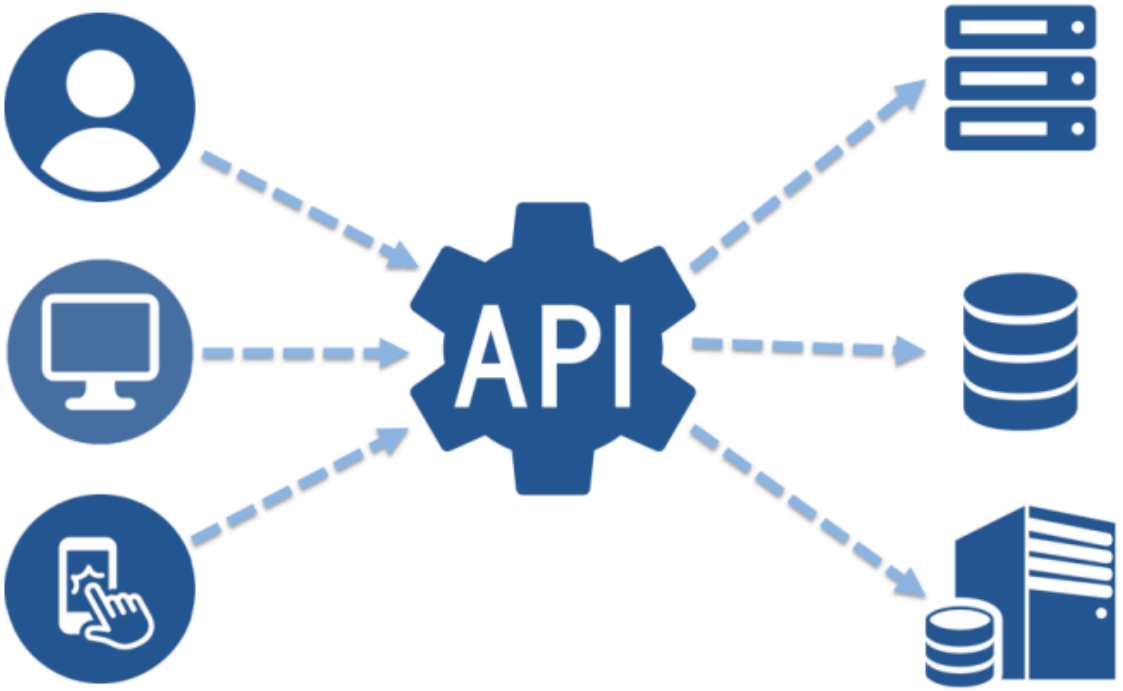
Anàlisi de Rendiment i Optimització d'Apache HTTPD amb JMeter



Esteve Amargant Blanco
Josep Escoda
Desplegament d'aplicacions web

Utilitzarem el nostre servidor d'Apache que tenim a una màquina virtual, on instal·larem els mòduls necessaris per poder executar PHP. Alhora al nostre ordinador amfitrió descarregarem el programa JMeter (Màquina Virtual de Java necessària).

Preparació del Servidor Apache (VM)

1.1. Creació d'un recurs estàtic

Per aïllar el cost de la connexió TCP del temps de processament de la CPU (ex. PHP), es fa servir un fitxer HTML estàtic:

```
echo "<h1>Test KeepAlive</h1>" | sudo tee /var/www/html/test.html
```

Comprova des de la consola de la VM que funciona correctament:

```
curl -i http://localhost/test.html
```

```
daw@MSI:~$ echo "<h1>Test KeepAlive</h1>" | sudo tee /web/test.html
<h1>Test KeepAlive</h1>
daw@MSI:~$ curl -i http://localhost:8080/test.html
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2026 12:49:40 GMT
Server: Apache
Last-Modified: Sun, 18 Oct 2026 12:49:34 GMT
ETag: "18-65e1cd44f57f2"
Accept-Ranges: bytes
Content-Length: 24
Content-Type: text/html

<h1>Test KeepAlive</h1>
daw@MSI:~$
```

1.2. Restricció de filtres d'Apache (MPM)

Per simular un entorn de saturació amb pocs usuaris de prova, limitarem el nombre de *workers* màxims d'Apache.

1. Edita el fitxer de configuració MPM (habitualment /etc/apache2/mods-available/mpm_prefork.conf):

```
<IfModule mpm_prefork_module>
    StartServers             2
    MinSpareServers          2
    MaxSpareServers          5
    MaxRequestWorkers       10
    MaxConnectionsPerChild   0
```

```
</IfModule>
```

```
  GNU nano 8.7.1
# prefork MPM
# StartServers: number of server processe
# MinSpareServers: minimum number of serv
# MaxSpareServers: maximum number of serv
# MaxRequestWorkers: maximum number of se
# MaxConnectionsPerChild: maximum number

StartServers            2
MinSpareServers         2
MaxSpareServers         5
MaxRequestWorkers       10
MaxConnectionsPerChild  0
```

2. Activa el mòdul si no ho està i reinicia el servei:

```
sudo a2dismod mpm_event mpm_worker && sudo a2enmod mpm_prefork
sudo systemctl restart apache2
```

```
daw@MSI:~$ sudo a2dismod mpm_event
Module mpm_event already disabled
daw@MSI:~$   systemctl restart apache2
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to restart 'apache2.service'.
Multiple identities can be used for authentication:
 1.  esteve
 2.  DAW,,, (daw)
Choose identity to authenticate as (1-2): 2
Password:
==== AUTHENTICATION COMPLETE ====
Job for apache2.service failed because the control process exited with error code.
See "systemctl status apache2.service" and "journalctl -xeu apache2.service" for details.
daw@MSI:~$ sudo a2dismod mpm_event
```

```
daw@MSI:~$ sudo a2dismod mpm_event
Module mpm_event already disabled
daw@MSI:~$ sudo a2enmod mpm_prefork
Considering conflict mpm_event for mpm_prefork:
Considering conflict mpm_worker for mpm_prefork:
Enabling module mpm_prefork.
To activate the new configuration, you need to run:
  systemctl restart apache2
daw@MSI:~$ sudo apache2ctl configtest && sudo systemctl restart apache2
apache2ctl -V | grep -i mpm
Syntax OK
Server MPM:     prefork
```

1.3. Simulació de latència de xarxa

VirtualBox té una latència pràcticament nul·la (< 1 ms). Afegirem 100 ms de latència artificial a la interfície de xarxa de la VM per simular una connexió WAN real.

1. Identifica la teva interfície de xarxa .

```
daw@MSI:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 10.255.255.254/32 ::1/128
eth0             UP             172.27.243.187/20 fe80::215:5dff:fef9:a3a7/64
daw@MSI:~$
```

2. Aplica la regla de latència:

```
sudo tc qdisc add dev enp0s3 root netem delay 100ms
```

```
daw@MSI:~$ sudo tc qdisc add dev eth0 root netem delay 100ms
tc qdisc show dev eth0
qdisc netem 8001: root refcnt 65 limit 1000 delay 100ms seed 4212482584139125865
daw@MSI:~$
```

3. Comprova des de la màquina amfitriona fent un ping cap a la IP de la VM. El temps de resposta ha de ser d'uns **100 ms**.

```
Microsoft Windows [Versión 10.0.26300.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Estev>ping 172.27.243.187

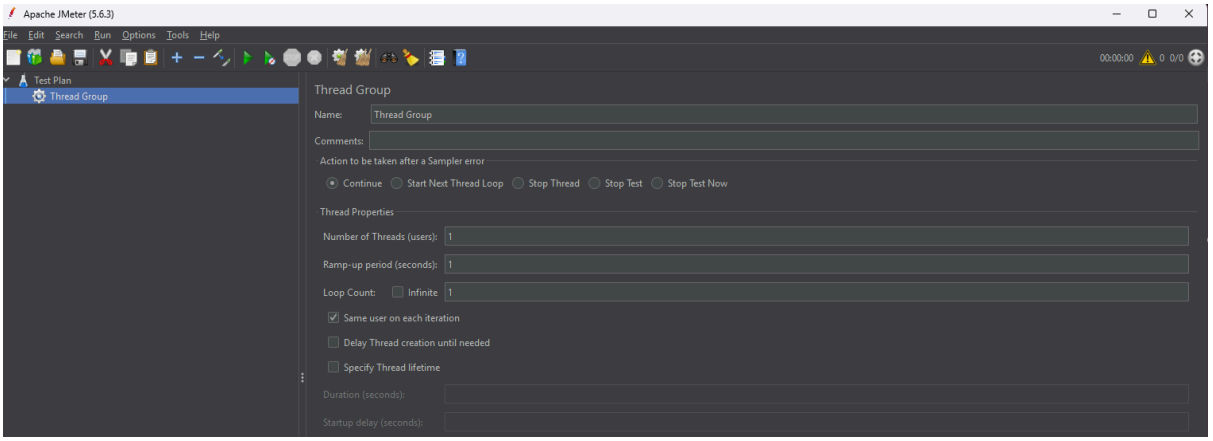
Haciendo ping a 172.27.243.187 con 32 bytes de datos:
Respuesta desde 172.27.243.187: bytes=32 tiempo=100ms TTL=64
Respuesta desde 172.27.243.187: bytes=32 tiempo=100ms TTL=64
Respuesta desde 172.27.243.187: bytes=32 tiempo=100ms TTL=64
Respuesta desde 172.27.243.187: bytes=32 tiempo=100ms TTL=64

Estadísticas de ping para 172.27.243.187:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 100ms, Máximo = 100ms, Media = 100ms

C:\Users\Estev>
```

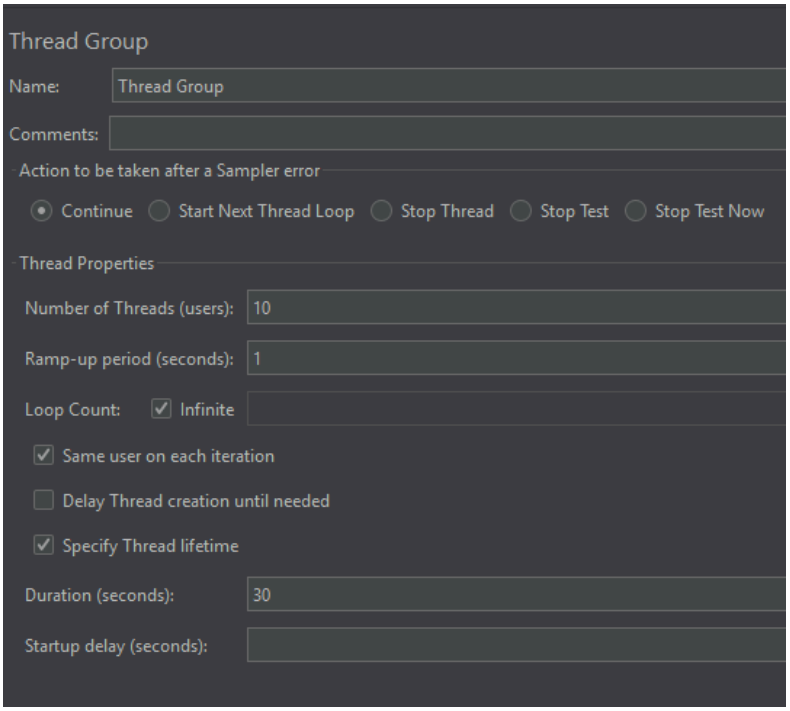
Configuració del Pla de Proves a JMeter (Amfitrió)

Obre Apache JMeter a la màquina amfitriona i configura l'arbre de la prova (afegir al menú esquerre):



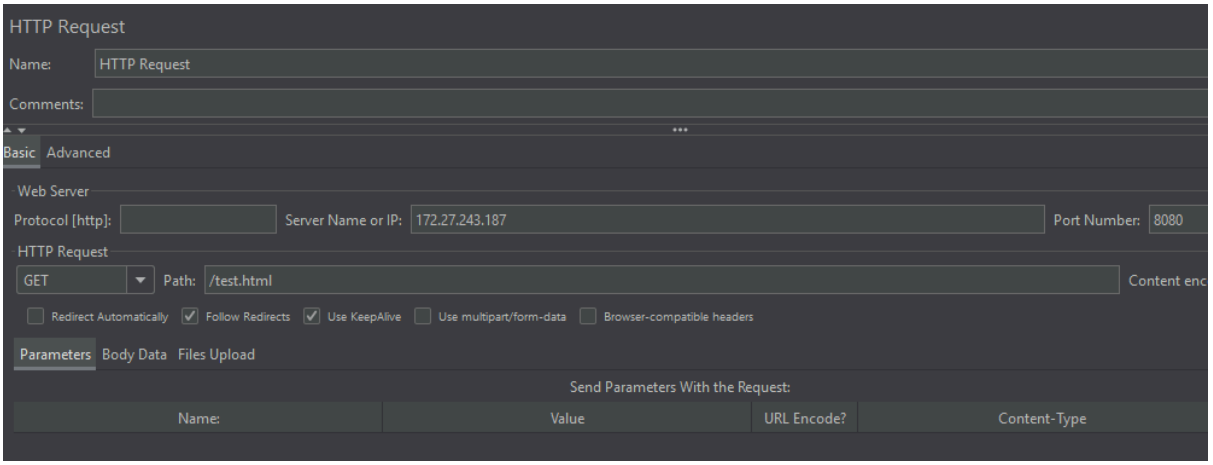
2.1. Thread Group (Grup d'Usuaris)

- Number of Threads (users): 10 *(ajustat exactament al nombre de MaxRequestWorkers per evitar cues de saturació)*.
- Ramp-up period (seconds): 1
- Loop Count: Marca la casella Infinite.
- Specify Thread duration: Marca la casella i indica 30 segons (la prova durarà 0.5 minut exactament).



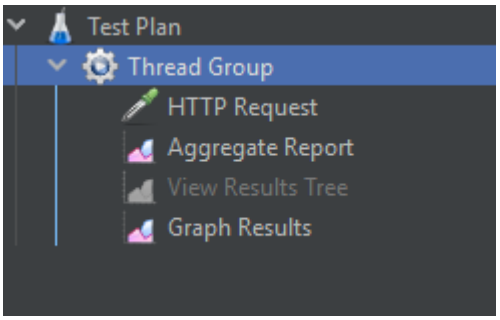
2.2. HTTP Request Sampler

- Afegir dins del Thread Group: Add → Sampler → HTTP Request.
- Server Name or IP: IP de la VM.
- HTTP Details: Mètode GET, Path: /test.html.
- Use Keep-Alive: Marca la casella (*caldrà desmarcar-la al segon test!*)



2.3. Receptors (Listeners)

- Afegir els següents receptors:
  - Add → Listener → Aggregate Report (el que utilitzareu realment)
  - Add → Listener → View Results Tree (desactiva'l o usa'l només per depurar, ja que consumeix molta memòria RAM durant la prova de càrrega).
  - Add → Listener → Graph Results



Execució dels tests

Executaràs la mateixa prova de JMeter (30 segons, 10 usuaris) sota 2 configuracions diferents d'Apache. Després de cada canvi a /etc/apache2/apache2.conf, cal reiniciar el servei: `sudo systemctl restart apache2`. I **Netejar** el *listener*.

Test A: KeepAlive On

Configuració a apache2.conf:

Servidor amb KeepAlive On

- A la VM, edita /etc/apache2/apache2.conf i assegura la directiva:

  ```
  KeepAlive On
  KeepAliveTimeout 15
  ```

```
#
# KeepAlive: Whether or not to allow persistent connections (more than
# one request per connection). Set to "Off" to deactivate.
#
KeepAlive On

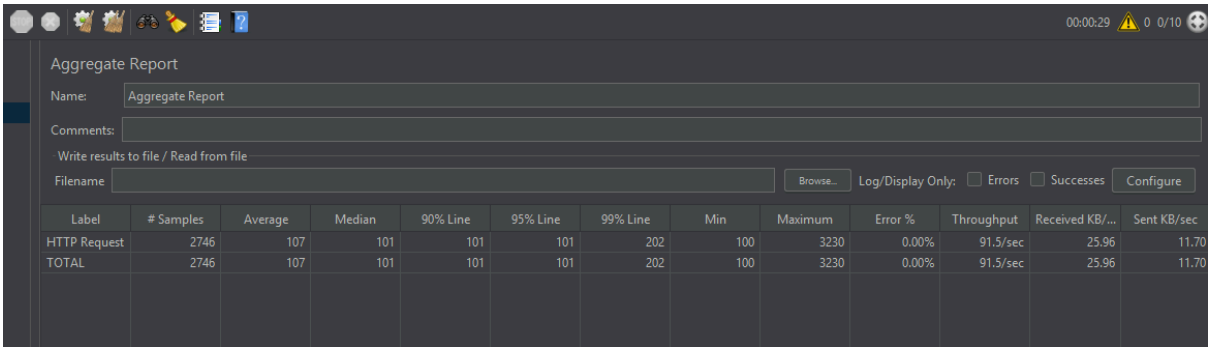
#
# MaxKeepAliveRequests: The maximum number of requests to allow
# during a persistent connection. Set to 0 to allow an unlimited amount.
# We recommend you leave this number high, for maximum performance.
#
MaxKeepAliveRequests 100

#
# KeepAliveTimeout: Number of seconds to wait for the next request from the
# same client on the same connection.
#
KeepAliveTimeout 15
```

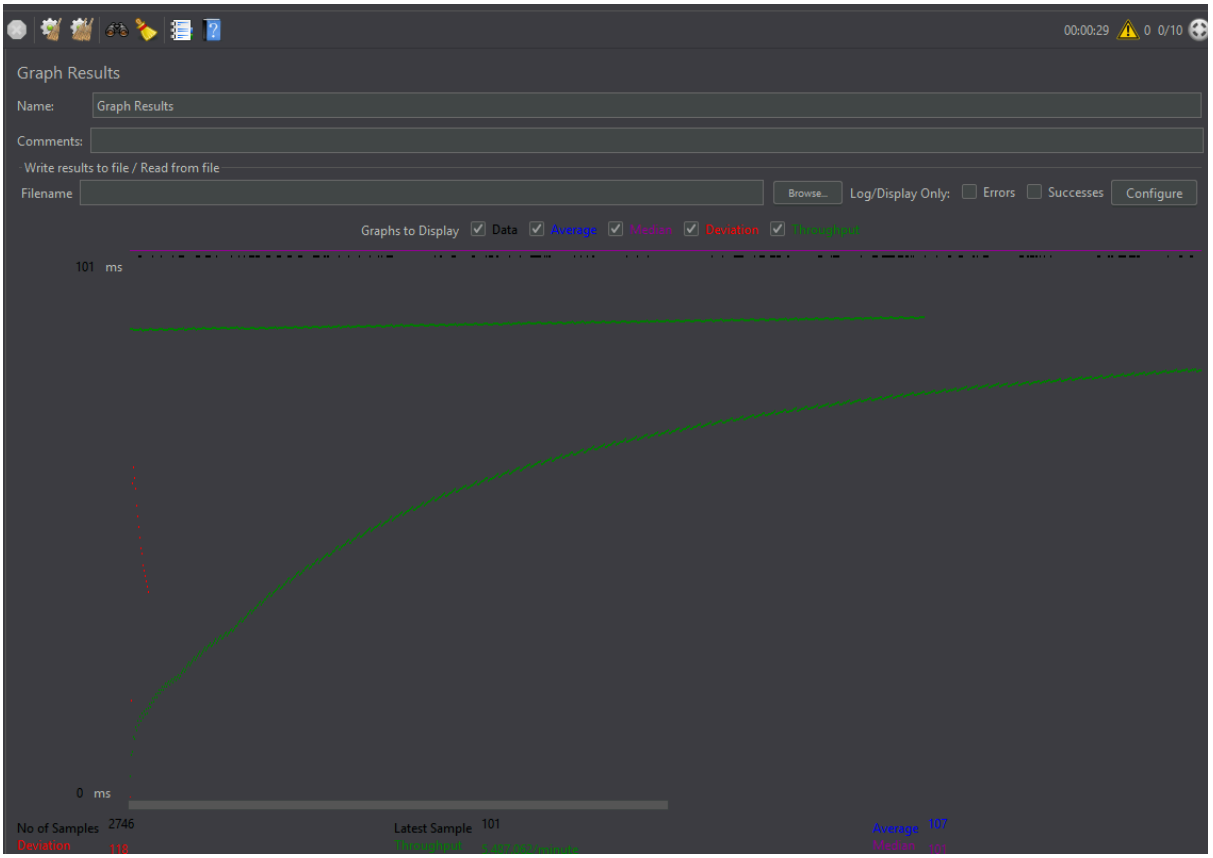
- Reinicia Apache: sudo systemctl restart apache2

```
daw@MSI:~$ sudo apache2ctl configtest && sudo systemctl restart apache2
grep -E "^KeepAlive" /etc/apache2/apache2.conf
Syntax OK
KeepAlive On
KeepAliveTimeout 15
daw@MSI:~$
```

- A JMeter, esborra els resultats previs (menú contextual) i executa la prova durant 30 segons.



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
HTTP Request	2746	107	101	101	101	202	100	3230	0.00%	91.5/sec	25.96	11.70
TOTAL	2746	107	101	101	101	202	100	3230	0.00%	91.5/sec	25.96	11.70



Test B: KeepAlive Off (Connexions individuals)

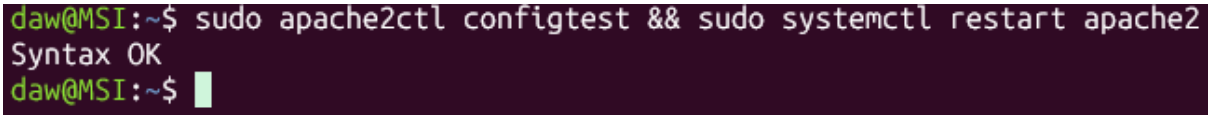
- A la VM, edita /etc/apache2/apache2.conf i canvia la directiva:

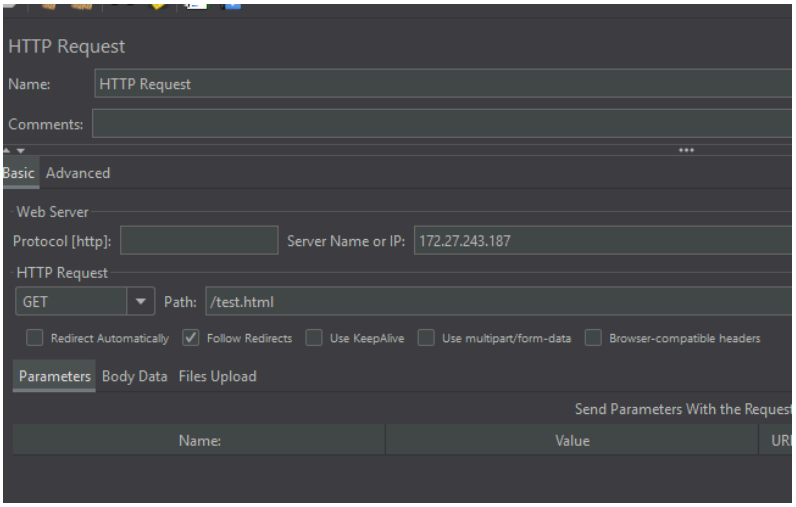
  `KeepAlive Off`

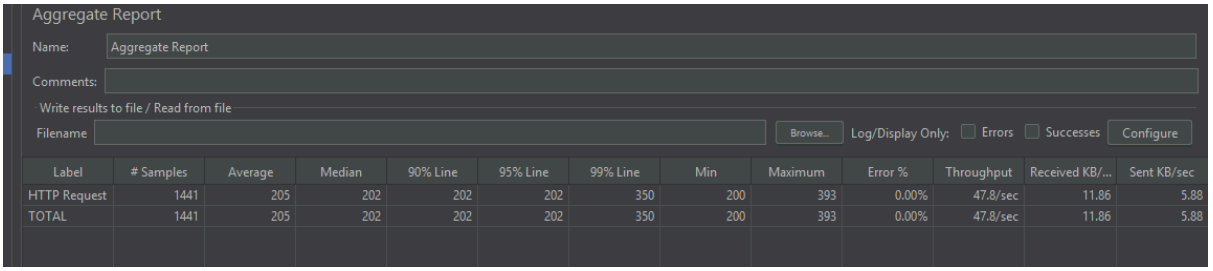
```
#
# KeepAlive: Whether or not to allow persistent connections (more than
# one request per connection). Set to "Off" to deactivate.
#
KeepAlive Off
```

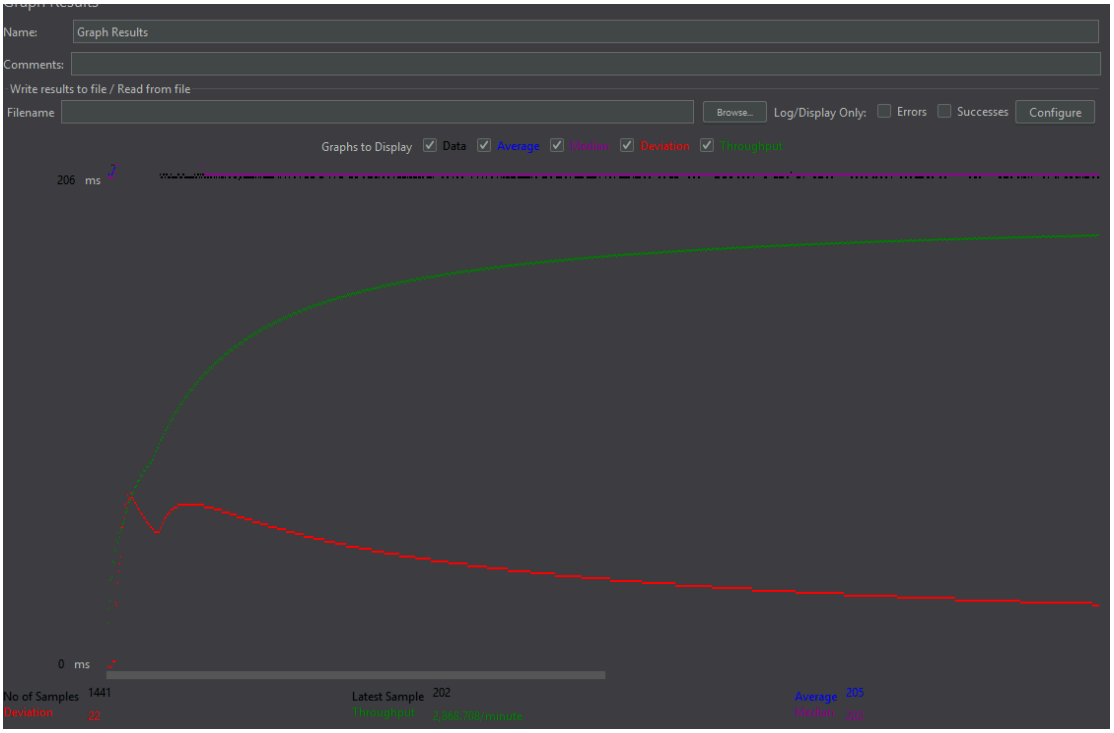
Reinicia Apache: sudo systemctl restart apache2

- A JMeter, esborra les dades anteriors, desactiva el *KeepAlive* del *HTTPRequest, ja que* si no JMeter força a mantenir les connexions obertes, i executa la prova durant 30 segons.









Recollida i Anàlisi de Dades

Emplena la següent taula amb les dades obtingudes del panell **Aggregate Report** de JMeter al final de cada execució de 60 segons:

Mètrica	Test A (KeepAlive On)	Test B (KeepAlive Off)
# Samples (Peticions totals)	**2746**	**1441**
Average (Temps de resposta mitjà - ms)	**107**	**205**
95% Line (ms)	**101**	**202**
Throughput (Peticions / segon)	**91,5**	**47,8**
Error % (Peticions fallides)	**0.00%**	**0.00%**

Respon a les següents qüestions

Respon a les següents preguntes en el teu informe de la pràctica basant-te en les dades obtingudes:

1. **Per què el temps mitjà de resposta es duplica exactament en desactivar KeepAlive?**

Per que amb el KeepAlive On la conexio TCP esta oberta i al fer el get la resposta torna, en canvi amb el OFF cada peticio ha de obrir la seva propia conexxio i la resposta torna mes tard

2. **Quin paper juga el mòdul mpm_prefork en aquest experiment respecte a mpm_event?**

Cada connexió te un procés sencer, sense fils. Com que hem posat que tinguem maxim 10 conecions obertes, si hi hagues una 11 conexio hauria de esperar que alguna de les 10 acabes. npm_fork tambe fa que cada proces quedi lligat a un client mentres la conexxio estigui oberta.

3. **En quins escenaris reals és recomanable desactivar el KeepAlive o reduir dràsticament el seu timeout?**

Quant tens moltes conexxions i aquestes nomes fan poques peticions, o Apis que reben una peticio per client nomes.
Servidors que nomes tenen una peticio per descarregar fitxers.

Neteja de l'entorn:

- Eliminem la latència de xarxa introduïda amb tc a la VM:

```
sudo tc qdisc del dev <la_teva_interfície> root
```

